ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТ №1
ТАЙЛАН

Үйлдлийн систем /F.CSM302/

Сэдэв:	Sudoku thread validator Судалгаа, хэрэгжилт, туршилтын тайлан
Сурах бичиг:	Operating System Concepts, 4-р бүлэг
Хуваарилалтын код:	IV-2-5
Хичээл заасан багш:	Г.Ганбат /F.CS10/
Гүйцэтгэсэн баг:	Э.Сумъяасүрэн /B200900040/ Т.Мөнгөнтүлхүүр /B241910041/ Э.Тэмүүлэн /B231910037/ А.Марал-Эрдэнэ /B241940106/ Б.Ундрах /B221910803/ Ө.Зоригтбаатар /B242270078/

Улаанбаатар хот, 2026 он

Хураангуй

Асуудал. 9×9 хэмжээтэй Sudoku-гийн шийдэл зөв байхын тулд 9 мөр, 9 багана, 9 ширхэг 3×3 дэд тор буюу нийт 27 бүс тус бүрд 1–9 цифр яг нэг удаа орсон байх ёстой. Бүсүүд хоорондоо хамааралгүй тул энэ шалгалт нь өгөгдлийн зэрэгцээ (data parallelism) ажлын сонгодог жишээ болно. Сурах бичгийн [1] Бүлэг 4-ийн Programming Project 1 нь энэ шалгалтыг олон урсгалаар (thread) гүйцэтгэх программ бичихийг шаарддаг.

Шийдэл. Бид C11 хэл, POSIX Pthreads санг ашиглан шалгагч хөдөлгүүр бүтээв. Хөдөлгүүр 1, 11, 27 урсгалын гурван стратегитай. Эх урсгал `parameters` бүтцийг `pthread_create()`-ээр дамжуулж, worker урсгал бүр зөвхөн өөрийн `valid[i]` нүдэнд үр дүнгээ бичнэ. Эх урсгал `pthread_join()`-оор бүгдийг хүлээж, эцсийн шийдвэрийг гаргана. Хөдөлгүүрийг вэб хөтөчөөс ашиглах боломжтой болгохын тулд зөвхөн localhost дээр ажиллах Python HTTP гүүр, засварлах боломжтой Dashboard болон CPU-ийн хуваарийг загварчилсан сургалтын симулятор нэмж хийсэн.

Туршилт. Функциональ зөв байдлыг 2170 автомат шалгалтаар (файлын тест, бүсийн тусгаарлалт, 300 санамсаргүй самбарын стресс тест, ThreadSanitizer ба AddressSanitizer+UBSan) баталгаажуулав. Гүйцэтгэлийг 2 vCPU бүхий Linux орчинд гурван түвшинд хэмжив: (i) нэг процесс доторх 2000 давталт, (ii) процесс бүрийг шинээр эхлүүлсэн 300 давталт, (iii) HTTP-ээр дамжсан төгсгөл-хоорондын хугацаа. Мөн `taskset` ашиглан 1 ба 2 цөмийн нөхцөлийг харьцуулав.

Гол үр дүн. Бүх 2170 шалгалт амжилттай болж, өгөгдлийн уралдаан (data race) болон санах ойн алдаа илрээгүй. Гэвч урсгалын тоо нэмэгдэх тусам хугацаа *өссөн*: процесс доторх дундаж хугацаа 1, 11, 27 урсгалд харгалзан 35.1, 253.0, 649.0 µs байв. Урсгал бүр ойролцоогоор 15–24 µs нэмэлт зардал авчирдаг бол 243 нүдийг бүхэлд нь шалгах ажил 9 µs-ээс бага хугацаа зарцуулдаг. Иймд энэ хэмжээний ажилд урсгал үүсгэх зардал давамгайлж, зэрэгцээ гүйцэтгэлийн ашиг гарахгүй гэдгийг Амдалын хууль болон шугаман зардлын загвараар тайлбарлав. Түүнчлэн 11 урсгалын стратегид ачааллын тэнцвэргүй байдал (81 ба 9 нүд) байгааг симулятор болон онолын шинжилгээгээр харуулав.

Түлхүүр үг: урсгал, POSIX Pthreads, fork–join, өгөгдлийн зэрэгцээ, Амдалын хууль, ThreadSanitizer, Sudoku.

Гарчиг

Хураангуй . . . . . **i**

1 Удиртгал **1**

1.1 Зорилго 1

1.2 Ач холбогдол 1

1.3 Хамрах хүрээ 1

1.4 Тайлангийн бүтэц 2

2 Сурах бичгийн шаардлага **2**

2.1 Даалгаврын агуулга 2

2.2 Шаардлагын хяналтын матриц 2

3 Онолын үндэс **2**

3.1 Процесс ба урсгал 2

3.2 Зэрэгцээ ажиллагаа: concurrency ба parallelism 3

3.3 Fork–join загвар ба Pthreads API 3

3.4 Санах ойн харагдах байдал ба өгөгдлийн уралдаан 3

3.5 Шалгах алгоритм ба түүний зөв байдал 4

3.6 Гүйцэтгэлийн онолын загвар 5

4 Системийн зохиомж **5**

4.1 Ерөнхий архитектур 5

4.2 Бүрэлдэхүүн хэсгүүд 6

4.3 C хөдөлгүүрийн дотоод бүтэц 6

4.4 Өгөгдлийн бүтэц 7

4.5 HTTP гүүрийн дараалал ба аюулгүй байдал 7

5 Хэрэгжилт **8**

5.1 Хэрэгсэл, хэл, сан 8

5.2 Параметр дамжуулах бүтэц 8

5.3 Бүс шалгах функц 8

5.4 Worker функцууд ба төлөвлөгөө 9

5.5 Урсгал үүсгэх ба хүлээх 10

5.6 Оролт, командын мөрийн интерфэйс 11

5.7 HTTP гүүрийн гол хэсэг 11

6 Туршилт ба UI дүрслэл **11**

6.1 Dashboard: бодит хөдөлгүүрийн интерфэйс 12

6.2 Юуг өөрчилж, юуг ажиглах вэ 12

6.3 Алдаатай ба бүрэн бус самбар 13

6.4 Сургалтын симулятор 13

7 Туршилтын арга **15**

7.1 Туршилтын орчин 15

7.2 Туршилтын өгөгдөл 15

7.3 Туршилтын төлөвлөгөө 16

7.4 Хэмжилтийн аргачлал 16
8 Үр дүн 17
8.1 Функциональ зөв байдал (E1–E4) 17
8.2 Дулаан хэмжилт ба цөмийн тооны нөлөө (E5, E7) 18
8.3 Хүйтэн хэмжилт (E6) 19
8.4 Нэг урсгалын нэмэлт зардлын үнэлгээ 19
8.5 HTTP-ээр дамжсан төгсгөл-хоорондын хугацаа (E8) 20
8.6 Симуляторын загварын үр дүн (E9) 21
9 Хэлэлцүүлэг 22
9.1 Яагаад олон урсгал удаан байв? 22
9.2 Яагаад нэг цөм хурдан байв? 22
9.3 Хүйтэн ба дулаан хэмжилтийн ялгаа 22
9.4 11 урсгалын стратегийн ачааллын тэнцвэргүй байдал 23
9.5 Симулятор ба бодит хэмжилтийн зөрүү 23
9.6 Хязгаарлалт 23
9.7 Сайжруулах санал 23
10 Дүгнэлт 24
Ашигласан материал 25
Хавсралт А. Ажлын төлөвлөгөө 26

Зургийн жагсаалт

4.1 Системийн архитектур ба өгөгдлийн урсгал 6
4.2 11 урсгалын горимын fork–join диаграмм (схемчилсэн, масштабгүй) 7
4.3 27 урсгалын горимд гурван worker-ийн хариуцах бүс ба `parameters` утга . . 8
6.1 Dashboard: зөв самбарыг 11 урсгалаар шалгасан үр дүн (бодит дэлгэцийн зураг) 12
6.2 Dashboard: нэг нүд буруу самбарыг 27 урсгалаар шалгаж, T19 урсгалын бүсийг тодруулсан нь 13
6.3 Dashboard: 3 хоосон нүдтэй бүрэн бус самбар (11 урсгал). Хоосон нүд бүр буруу гэж тооцогдоно 14
6.4 Симулятор: 11 урсгалын горимын 40-р tick. Дэд торын worker-ууд дуусч `valid[2..6]=1` болсон, T0 ба T1 үргэлжилж байна 14
6.5 Симуляторын CPU цөмийн хуваарь (4 цөм, $c = 3$, квант 8): дээр 11 урсгал (99 tick), доор 27 урсгал (91 tick) 14
8.1 Дулаан хэмжилт: урсгалын тоо ба цөмийн тооны нөлөө (алдааны зураас: ± 1 стандарт хазайлт) 18
8.2 Хүйтэн хэмжилтийн медиан (алдааны зураас: 25 ба 75-р перцентиль) 19
8.3 Хугацаа урсгалын тооноос шугаман хамаарч байгаа нь (шулуун: хамгийн бага квадратын тааруулга) 20
8.4 HTTP хүсэлтийн хугацааны бүрэлдэхүүн (медиануудын зөрүүгээр ойролцоолсон) 20
8.5 Dashboard-ын гүйцэтгэлийн хэмжилт ба шалгалтын түүх (бодит дэлгэцийн зураг) 21
8.6 Симулятор: цөмийн тооноос хамаарах нийт хугацаа ($c = 3$ tick, квант 8, FIFO) 21

Хүснэгтийн жагсаалт

1.1 Төслийн хамрах хүрээ 1
2.1 Сурах бичгийн шаардлага ба хэрэгжилтийн уялдаа 3
3.1 Ашигласан POSIX функцууд 4
3.2 Стратеги бүрийн ажлын хуваарилалт 5
4.1 Бүрэлдэхүүн хэсгүүд ба тэдгээрийн үүрэг 6
4.2 Гол өгөгдлийн бүтэц ба хуваалцлын дүрэм 7
5.1 Хэрэгжилтэд ашигласан технологи 8
5.2 Командын мөрийн сонголтууд 11
6.1 Интерфэйс дээрх туршилтын хувьсагч ба ажиглалт 12
7.1 Туршилтын орчин 15
7.2 Тестийн оролтын файлууд ба хүлээгдэх үр дүн 15
7.3 Туршилтууд, арга, хүлээгдэх үр дүн 16
8.1 Автомат тестийн нэгтгэл (`make test`, нийт 6.5 секунд) 17
8.2 27 урсгалын горимын `valid[]` массив (мөр 1–9 | багана 1–9 | дэд тор 1–9) . 17

8.3 Нэг процесс доторх хугацаа, `-b 2000`, 3 ажиллуулалтын дундаж $\pm$ стандарт хазайлт (µs) 18
8.4 Процесс бүрийг шинээр эхлүүлсэн хэмжилт, горим бүрд $n = 300$ (µs) 19
8.5 $T(n) \approx a + b\,n$ шугаман загварын коэффициент 20
8.6 HTTP хүсэлтийн хугацаа, горим бүрд 100 хүсэлт (2 цөм) 20
8.7 Симуляторын үр дүн, 4 цөм, $c = 3$, квант 8, FIFO 21

1. Удиртгал

1.1 Зорилго

Энэхүү бие даалтын үндсэн зорилго нь үйлдлийн системийн *урсгал (thread)*-ын ойлголтыг онол төдийгүй практикаар эзэмших явдал юм. Үүнд дараах зорилтуудыг дэвшүүлэв.

(1) Сурах бичгийн Sudoku шалгагчийг POSIX Pthreads-ээр бүрэн хэрэгжүүлэх: урсгал үүсгэх, параметр дамжуулах, үр дүн буцаах, хүлээх.

(2) 1, 11, 27 урсгалын стратегийг нэг программд хэрэгжүүлж, харьцуулах боломжтой болгох.

(3) Хэрэгжилтийн зөв байдлыг автомат тест болон sanitizer хэрэгслээр нотлох.

(4) Урсгалын тоо, CPU цөмийн тоо гүйцэтгэлд хэрхэн нөлөөлөхийг бодит хэмжилтээр судалж, онолын загвартай харьцуулах.

(5) Үр дүнг ойлгомжтой харуулах вэб интерфэйс болон сургалтын симулятор бүтээх.

1.2 Ач холбогдол

Орчин үеийн процессорууд олон цөмтэй тул программыг зэрэгцээ ажиллуулах чадвар нь гүйцэтгэлийн гол нөөц болсон. Гэвч урсгал бүр үнэ төлбөргүй биш: түүнийг үүсгэх, хуваарьт оруулах, хүлээх, кэшийн өгөгдлийг цөм хооронд шилжүүлэх зэрэгт цаг зарцуулагдана. Sudoku шалгалт нь ажил нь маш бага (243 нүд унших) учраас урсгалын *нэмэлт зардлыг* тусгаарлан хэмжихэд тохиромжтой туршилтын орчин болдог. Иймд энэ төсөл нь «хэдэн урсгал хэрэглэх вэ?» гэсэн практик асуултад хэмжилтэд суурилсан хариулт өгөх ач холбогдолтой.

1.3 Хамрах хүрээ

Хүснэгт 1.1. Төслийн хамрах хүрээ

Хамрагдсан	Хамрагдаагүй
9×9 бүрэн шийдлийг шалгах (хоосон нүд = буруу)	Sudoku бодох (solver) алгоритм
1 / 11 / 27 урсгалын стратеги, POSIX Pthreads	Windows Threads API, OpenMP, урсгалын сан (pool)
Файл, stdin, санамсаргүй үүсгэгч, алдаа оруулагчаар ороолт бэлтгэх	N×N (16×16, 25×25) ерөнхий хэмжээ
Автомат тест, TSan, ASan+UBSan	Формаль баталгаажуулалт (model checking)
Linux 2 vCPU орчны бодит хэмжилт, 1 ба 2 цөмийн харьцуулалт	Олон төрлийн техник хангамж дээрх харьцуулалт
Localhost HTTP гүүр, Dashboard, сургалтын симулятор	Интернэтэд нээлттэй үйлчилгээ, нэвтрэлт

1.4 Тайлангийн бүтэц

2-р бүлэгт сурах бичгийн шаардлагыг, 3-р бүлэгт онолын үндсийг, 4-р бүлэгт системийн зохиомжийг, 5-р бүлэгт хэрэгжилтийг тайлбарлана. 6-р бүлэг интерфэйс дээр юуг өөрчилж, юуг ажиглахыг, 7-р бүлэг туршилтын аргачлалыг, 8-р бүлэг үр дүнг, 9-р бүлэг тэдгээрийн тайлбар ба хязгаарлалтыг агуулна. 10-р бүлэгт дүгнэлт хийнэ.

2. Сурах бичгийн шаардлага

2.1 Даалгаврын агуулга

Silberschatz, Galvin, Gagne нарын *Operating System Concepts* (10-р хэвлэл) номын 4-р бүлэг «Threads & Concurrency»-ийн төгсгөлд байрлах Programming Project 1 нь Sudoku-гийн шийдэл зөв эсэхийг олон урсгалтай программаар шалгахыг даалгадаг [1]. Номын Figure 4.26-д зөв шийдлийн жишээ өгөгдсөн бөгөөд бид түүнийг программын анхдагч оролт болгон ашигласан. Сурах бичиг дараах стратегийг санал болгоно: бүх баганыг шалгах 1 урсгал, бүх мөрийг шалгах 1 урсгал, 3×3 дэд тор бүрийг шалгах 9 урсгал, нийт 11 урсгал. Мөн мөр, багана бүрт тусдаа урсгал үүсгэж урсгалын тоог нэмэгдүүлж болохыг дурдсан. Энэ нь манай 27 урсгалын стратегид харгалзана.

Даалгавар нь хоёр механизмыг тусгайлан шаарддаг. Нэгдүгээрт, эх урсгал worker бүрт шалгах байрлалыг (мөр, багана) агуулсан бүтцийг `pthread_create()`-ийн сүүлийн аргументаар дамжуулна. Хоёрдугаарт, бүх урсгалд харагдах бүхэл тоон массив үүсгэж, i-р worker өөрийн бүс зөв бол i-р элементэд 1 бичнэ. Эх урсгал бүх worker дуусахыг хүлээсний дараа массивыг шалгаж эцсийн дүгнэлт гаргана.

2.2 Шаардлагын хяналтын матриц

Шаардлага бүрийг кодын аль хэсэг хангаж, ямар туршилтаар баталгаажсаныг Хүснэгт 2.1-д үзүүлэв.

3. Онолын үндэс

3.1 Процесс ба урсгал

Процесс нь ажиллаж буй программын хамгаалагдсан орчин бөгөөд өөрийн хаягийн орон зай, нээлттэй файл, нөөцтэй. Урсгал нь процесс доторх CPU-д хуваарилагдах хамгийн жижиг нэгж юм. Нэг процессын урсгалууд код, глобал өгөгдөл, heap болон файлыг *хуваалцдаг*, харин урсгал бүр өөрийн регистрүүд, програмын тоолуур, стек болон урсгалын ID-тай байна [1]. Энэ хуваалцал нь өгөгдөл солилцоог хялбар болгодог ч хуваалцсан хувьсагч руу зэрэг бичих үед өгөгдлийн уралдаан үүсэх эрсдэлтэй.

Linux дээрх glibc-ийн NPTL хэрэгжилт нь «нэгийг-нэгд» (1:1) загвартай. Өөрөөр хэлбэл, `pthread_create()` дуудлага бүр `clone()` системийн дуудлагаар цөмийн шинэ task үүсгэдэг

Хүснэгт 2.1. Сурах бичгийн шаардлага ба хэрэгжилтийн уялдаа

ID	Шаардлага	Хэрэгжилт	Баталгаа	Төлөв
Ш1	9×9 тор, бүс бүрд 1–9 цифр яг нэг удаа	`check_region()`, bitmask	Тест 1–3	✓
Ш2	Бүх мөрийг шалгах 1 урсгал	`check_rows`, `count=9`	11 горим	✓
Ш3	Бүх баганыг шалгах 1 урсгал	`check_columns`, `count=9`	11 горим	✓
Ш4	3×3 дэд тор бүрд 1 урсгал (9)	`check_subgrid`	11/27 горим	✓
Ш5	Нийт 11 урсгал; нэмэлтээр илүү урсгал	`build_plan()`: 1/11/27	`-t` сонголт	✓
Ш6	`parameters` бүтцээр байрлал дамжуулах	`malloc` + `pthread_create` 4-р арг.	TSan, ASan	✓
Ш7	Урсгал бүрт харагдах `valid[]` массив, i-р worker → `valid[i]`	`static int valid[27]`	Тест 2 (valid[] мөр)	✓
Ш8	Эх урсгал бүгдийг хүлээж, массивыг шалгах	`pthread_join` давталт	Гаралтын код 0/1	✓
Н1	*Нэмэлт:* алдаатай нүдийг тодорхойлох	`worker_report`, `issue`	CLI, Dashboard	✓
Н2	*Нэмэлт:* гүйцэтгэл хэмжих	`-b`, `CLOCK_MONOTONIC`	8-р бүлэг	✓
Н3	*Нэмэлт:* вэб интерфэйс, симулятор	`server.py`, HTML	6-р бүлэг	✓

бөгөөд тэрхүү task-ыг цөмийн хуваарилагч бие даан CPU-д оноодог [9]. Иймд урсгал үүсгэх нь хэрэглэгчийн түвшний функц дуудлагаас хамаагүй үнэтэй: стек санах ой (анхдагч 8 MiB виртуал), хамгаалалтын хуудас, TLS бүтэц, цөмийн task бүтэц шаардагдана.

3.2 Зэрэгцээ ажиллагаа: concurrency ба parallelism

Concurrency нь хэд хэдэн ажил нэг хугацааны интервалд урагшилж байгааг, parallelism нь тэдгээр нь *яг нэгэн зэрэг* өөр өөр цөм дээр гүйцэтгэгдэж байгааг илэрхийлнэ [1]. Нэг цөмтэй системд 27 урсгал concurrency-тэй боловч parallelism-гүй. Sudoku шалгалтад бүс бүр бусдаасаа хамааралгүй тул ажлыг өгөгдлөөр нь хуваах (data parallelism) нь байгалийн шийдэл болно.

3.3 Fork–join загвар ба Pthreads API

Төслийн бүтэц нь сонгодог *fork–join* загвар юм: эх урсгал ажлыг хуваан worker-уудыг үүсгэнэ (fork), worker-ууд зэрэг ажиллана, эх урсгал бүгдийг хүлээж үр дүнг нэгтгэнэ (join). Ашигласан гол функцуудыг Хүснэгт 3.1-д үзүүлэв [2, 3].

3.4 Санах ойн харагдах байдал ба өгөгдлийн уралдаан

POSIX стандарт нь `pthread_create()` ба `pthread_join()` зэрэг функцуудыг санах ойг синхрончлох цэг гэж тодорхойлдог [2]. Энэ нь хоёр баталгаа өгнө: (1) эх урсгал `pthread_create()`-ээс өмнө бичсэн өгөгдлийг (`sudoku[][]`, `parameters`, тайлангийн нэр) шинэ урсгал заавал харна; (2) worker-ийн бичсэн `valid[i]`-г эх урсгал `pthread_join()` буцсаны дараа заавал харна.

Өгөгдлийн уралдаан гэдэг нь хоёр урсгал нэг санах ойн байршилд синхрончлолгүйгээр

Хүснэгт 3.1. Ашигласан POSIX функцууд

Функц	Үүрэг ба төсөл дэх хэрэглээ
`pthread_create(&tid, NULL, fn, arg)`	Шинэ урсгал үүсгэж `fn(arg)`-ийг ажиллуулна. `arg` нь `malloc`-оор үүсгэсэн `parameters*`.
`pthread_join(tid, NULL)`	`tid` урсгал дуусах хүртэл эх урсгалыг блоклоно. Дууссаны дараа `parameters`-ийг чөлөөлнө.
`pthread_exit(NULL)`	Worker урсгалыг дуусгана; үр дүнг `valid[]`-ээр дамжуулдаг тул буцаах утга хэрэглэхгүй.
`clock_gettime(CLOCK_MONOTONIC)`	Урсгал үүсгэхээс join дуусах хүртэлх хугацааг наносекундын нарийвчлалтай хэмжинэ.

хандаж, наад зах нь нэг нь бичих тохиолдол юм. Бернштейний нөхцөл [11] ёсоор хоёр ажил T_i, T_j-ийн унших (R) ба бичих (W) олонлог

$$W_i \cap W_j = \varnothing, \qquad R_i \cap W_j = \varnothing, \qquad W_i \cap R_j = \varnothing \tag{3.1}$$

нөхцөлийг хангавал тэдгээрийг зэрэг ажиллуулахад үр дүн өөрчлөгдөхгүй. Манай хэрэгжилтэд worker i-ийн бичих олонлог нь $W_i = \{\texttt{valid[i]},\ \texttt{reports[i]}\}$, унших олонлог нь $R_i = \{\texttt{sudoku}\}$ (зөвхөн уншина) тул (3.1) хангагдана. Иймээс mutex, атомик хувьсагч шаардлагагүй. Энэ дүгнэлтийг 8.1-р хэсэгт ThreadSanitizer-ээр туршилтаар баталгаажуулсан.

3.5 Шалгах алгоритм ба түүний зөв байдал

Бүс бүрийн 9 нүдийг $v_1, \dots, v_9$ гэвэл бид 16 битийн маск

$$M = \bigvee_{k=1}^{9} 2^{v_k} \tag{3.2}$$

тооцно. Бүс зөв байх зайлшгүй бөгөөд хүрэлцээтэй нөхцөл нь бүх $v_k \in \{1, \dots, 9\}$ ба

$$M = \sum_{d=1}^{9} 2^d = 2^{10} - 2 = 1022 = \texttt{0x3FE} \tag{3.3}$$

байх явдал юм. *Баталгаа:* $M = \texttt{0x3FE}$ бол 1–9 цифр бүр дор хаяж нэг удаа орсон. Нүд 9 ширхэг, ялгаатай цифр 9 тул Дирихлегийн зарчмаар цифр бүр *яг* нэг удаа орно. Эсрэгээр, давхардал байвал 9 нүдэнд 8-аас олонгүй ялгаатай цифр байх тул $M \neq \texttt{0x3FE}$ болно. □

Бүс бүрийг шалгах нь $\Theta(9)$ үйлдэл тул бүхэл самбарын нийт ажил $27 \times 9 = 243$ нүд унших үйлдэл болно. Энэ ажил стратегиас үл хамааран тогтмол. Стратеги нь зөвхөн ажлыг урсгалд хэрхэн хуваарилахыг өөрчилнө.

3.6 Гүйцэтгэлийн онолын загвар

p цөм дээр нийт W ажлын f хэсэг зэрэгцээ, $1-f$ хэсэг дараалсан бол Амдалын хуулиар [5] хурдатгал

$$S(p) = \frac{1}{(1-f) + f/p} \tag{3.4}$$

байна. Гэвч энэ томьёо урсгал үүсгэх зардлыг тооцдоггүй. Манай тохиолдолд эх урсгал n урсгалыг *дараалан* үүсгэдэг тул хугацааг ойролцоогоор

$$T(n) \;\approx\; a + b\,n + \max_{i} w_i \tag{3.5}$$

гэж загварчилж болно. Энд a нь тогтмол зардал, b нь нэг урсгалыг үүсгэж, хуваарьт оруулж, хүлээх зардал, w_i нь i-р worker-ийн ажил. Хэрэв $b \gg W/n$ бол урсгал нэмэх нь хугацааг *нэмэгдүүлнэ*. Энэ таамаглалыг 8-р бүлэгт хэмжилтээр шалгана.

Стратеги бүрийн ажлын хуваарилалтыг харьцуулбал (Хүснэгт 3.2) 11 урсгалын стратегид хамгийн урт worker 81 нүд шалгадаг. Иймд олон цөмтэй байсан ч эгзэгтэй зам (critical path) 81 нүднээс богиносохгүй, ачааллын тэнцвэргүй байдал үүснэ.

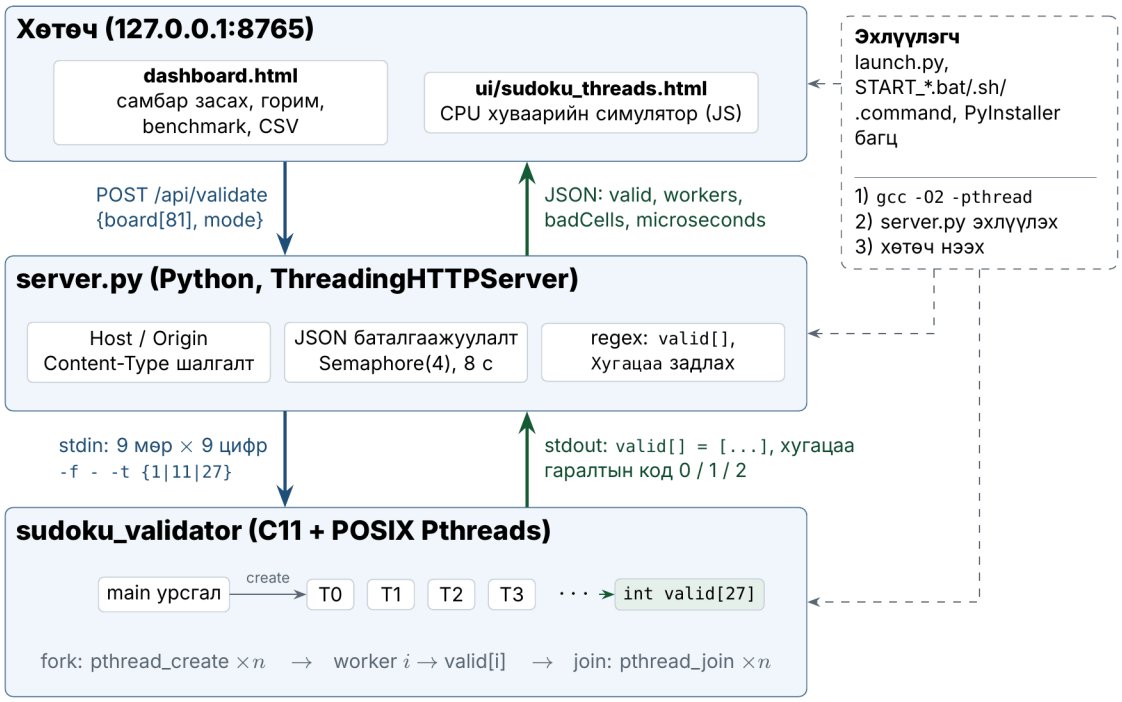
Хүснэгт 3.2. Стратеги бүрийн ажлын хуваарилалт

Стратеги	Урсгалын тоо	Worker-ийн ажил (нүд)	$\max w_i$	Тэнцвэр
`-t 1`	1	243	243	---
`-t 11`	11	81, 81, 9×9	81	Тэнцвэргүй
`-t 27`	27	27×9	9	Тэнцвэртэй

4. Системийн зохиомж

4.1 Ерөнхий архитектур

Систем нь гурван давхаргаас бүрдэнэ (Зураг 4.1): (1) хэрэглэгчийн интерфэйс болох хөтөч дээрх хоёр HTML хуудас; (2) зөвхөн `127.0.0.1:8765` дээр сонсдог Python HTTP гүүр; (3) шалгалтыг бодитоор хийдэг C11/Pthreads хөдөлгүүр. Хөдөлгүүр нь бие даасан командын мөрийн программ тул HTTP давхаргагүйгээр ч ажилладаг.



Зураг 4.1. Системийн архитектур ба өгөгдлийн урсгал

4.2 Бүрэлдэхүүн хэсгүүд

Хүснэгт 4.1. Бүрэлдэхүүн хэсгүүд ба тэдгээрийн үүрэг

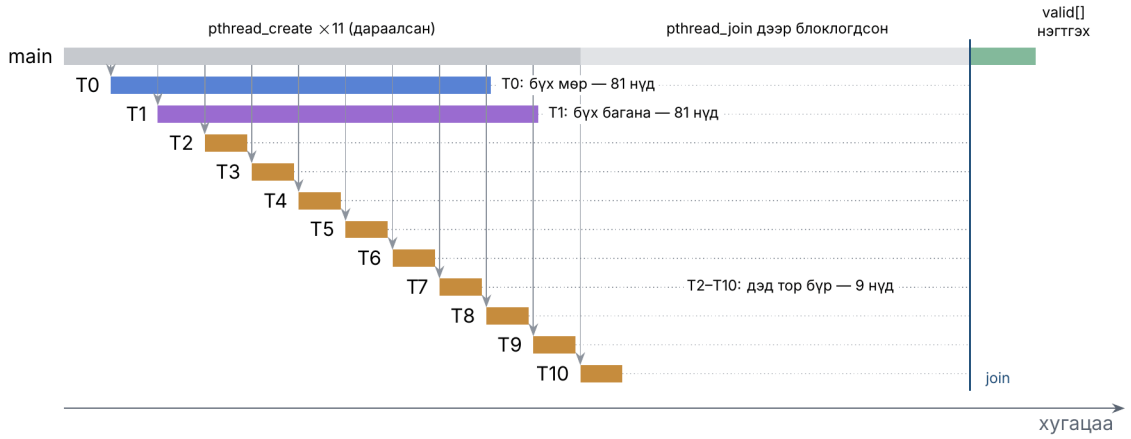
Файл	Хэл	Үүрэг
`sudoku_validator.c` (641 мөр)	C11	Оролт унших, урсгалын төлөвлөгөө, worker функцууд, оношлогоо, benchmark, CLI
`server.py` (162 мөр)	Python 3	Localhost HTTP сервер, оролт баталгаажуулах, C программыг дэд процессоор ажиллуулах
`dashboard.html`	HTML/JS	Засварлах самбар, горим сонгох, үр дүн, benchmark, CSV экспорт
`ui/sudoku_threads.html`	HTML/JS	Олон цөмт CPU, round-robin хуваарилагчийн tick-д суурилсан симулятор
`run_tests.sh`, `Makefile`	Bash, Make	Хөрвүүлэлт, sanitizer хувилбар, 2170 автомат шалгалт
`launch.py`, `START_*`	Python, скрипт	Нэг товшилтоор хөрвүүлэх, сервер эхлүүлэх, хөтөч нээх
`launch_standalone.py`	Python	PyInstaller-ээр Windows/macOS-ийн бэлэн программ болгох оролтын цэг

4.3 C хөдөлгүүрийн дотоод бүтэц

Хөдөлгүүрийн гүйцэтгэлийн дараалал: (1) `getopt`-оор аргумент задлах; (2) оролт бэлтгэх (файл, stdin, `-g` үүсгэгч, анхдагч Figure 4.26); (3) шаардлагатай бол `-e`, `-s`-ээр нүд өөрчлөх; (4) `build_plan()`-оор стратегийн дагуу `plan_item` массив үүсгэх; (5) `run_validation()`-оор урсгалуудыг үүсгэж, хүлээж, `valid[]`-ийг нэгтгэх; (6) тайлан хэвлэж, гаралтын код (0 зөв,

1 буруу, 2 алдаа) буцаах.

Зураг 4.2-д 11 урсгалын горимын fork–join дарааллыг дүрслэв. Эх урсгал урсгалуудыг нэг нэгээр нь үүсгэдэг тул үүсгэлт өөрөө дараалсан хэсэг болдог. Мөн T0, T1 нь бусдаасаа 9 дахин их ажил гүйцэтгэнэ.



Зураг 4.2. 11 урсгалын горимын fork–join диаграмм (схемчилсэн, масштабгүй)

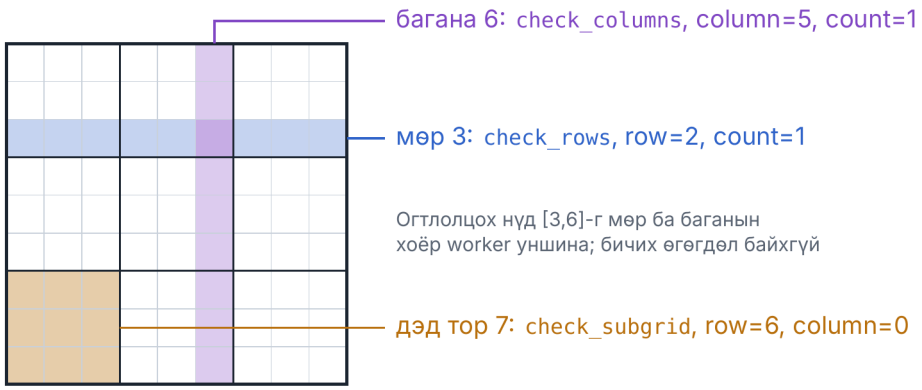
4.4 Өгөгдлийн бүтэц

Хүснэгт 4.2. Гол өгөгдлийн бүтэц ба хуваалцлын дүрэм

Бүтэц	Хэн бичнэ	Хэн уншина	Синхончлол / тайлбар
`int sudoku[9][9]`	main (create-ээс өмнө)	Бүх worker	`pthread_create` нь харагдах байдлыг баталгаажуулна
`parameters` (heap)	main	Тухайн нэг worker	Worker бүрт тусдаа `malloc`, join-ы дараа `free`
`int valid[27]`	Зөвхөн worker $i \rightarrow$ `valid[i]`	main (join-ы дараа)	Бичих олонлог давхцахгүй; `pthread_join`
`worker_report reports[27]`	main (нэр), worker i	main (join-ы дараа)	Нэрийг create-ээс өмнө бичнэ; асуудлыг worker бичнэ
`plan_item plan[27]`	main	main	Функцийн заагч, row, column, count

4.5 HTTP гүүрийн дараалал ба аюулгүй байдал

Dashboard самбарыг 81 бүхэл тоон массив болгон `POST /api/validate` хүсэлтээр илгээнэ. Сервер дараах шалгалтыг дараалан хийнэ: `Host` толгой нь `127.0.0.1:8765` эсвэл `localhost:8765` байх (DNS rebinding халдлагаас хамгаална); `Origin` нь мөн адил локал байх; `Content-Type` нь `application/json` байх; хэмжээ 1–8192 байт; самбар 81 ширхэг 0–9 бүхэл тоо; горим $\{1, 11, 27\}$. Нэг зэрэг ажиллах шалгалтыг `BoundedSemaphore(4)`-оор хязгаарлаж, C процессыг 8 секундын timeout-тай ажиллуулна. Статик файлын хувьд зөвхөн тодорхой жагсаалтад орсон хоёр HTML-ийг өгөх бөгөөд эх код, тестийн хавтсыг 404 хариугаар хаана. Эцсийн шийдвэр (`valid`) нь үргэлж C программын гаралтын кодоос



Зураг 4.3. 27 урсгалын горимд гурван worker-ийн хариуцах бүс ба `parameters` утга гарна.

5. Хэрэгжилт

5.1 Хэрэгсэл, хэл, сан

Хүснэгт 5.1. Хэрэгжилтэд ашигласан технологи

Технологи	Хэрэглээ
C11 (`-std=c11 -O2 -Wall -Wextra`)	Шалгагч хөдөлгүүр; анхааруулгагүй хөрвүүлэгдсэн [4]
POSIX Threads (glibc NPTL)	`pthread_create`, `pthread_join`, `pthread_exit`
`clock_gettime`, `getopt`, `strtoll`	Хугацаа хэмжих, CLI задлах, аргументыг чанд шалгах
GCC 13.3, GNU Make	Хөрвүүлэлт, `tsan`/`asan` зорилтот хувилбар
ThreadSanitizer, AddressSanitizer, UBSan	Өгөгдлийн уралдаан, санах ойн алдаа, тодорхойгүй үйлдэл илрүүлэх [6, 7, 10]
Python 3 стандарт сан	`http.server`, `subprocess`, `json`, `re`, `threading` [8]
HTML5, CSS, Vanilla JavaScript	Dashboard, SVG Gantt диаграмм, симулятор; гадны сангүй
PyInstaller, GitHub Actions	Windows EXE, macOS APP бэлэн багц

5.2 Параметр дамжуулах бүтэц

Сурах бичгийн `parameters` бүтцийг `count` ба `index` талбараар өргөтгөв. `count` нь нэг worker хэдэн дараалсан мөр/багана шалгахыг заадаг тул нэг функц 11 ба 27 горимд аль алинд ажиллана. `index` нь `valid[]` массив дахь байрлал.

5.3 Бүс шалгах функц

`check_region()` нь 3.5-р хэсгийн bitmask алгоритмыг хэрэгжүүлнэ. Бүсийн k-р нүдний координатыг `cell_of()` функц мөр, багана, дэд торын аль алинд нэг томьёогоор гаргадаг

Жагсаалт 1. Worker-т дамжуулах бүтэц ба хуваалцсан өгөгдөл

```c
typedef struct {
    int row;    /* шалгаж эхлэх мөр (0-ээс) */
    int column; /* шалгаж эхлэх багана (0-ээс) */
    int count;  /* дараалсан хэдэн мөр/багана шалгах */
    int index;  /* энэ worker-ийн valid[] массив дахь байрлал */
} parameters;

static int sudoku[N][N];                    /* worker-ууд зөвхөн уншина */
static int valid[MAX_WORKERS];              /* i-р worker зөвхөн valid[i]-д бичнэ */
static worker_report reports[MAX_WORKERS]; /* i-р worker зөвхөн reports[i]-д */
```

тул гурван төрлийн бүсэд нэг функц ашиглагдана. Алдаа илэрвэл `ISSUE_DUP` (давхардал), `ISSUE_RANGE` (хоосон эсвэл 1–9-өөс гадуур), `ISSUE_MISSING` (дутуу цифр) гэсэн оношлогоог тухайн worker-ийн өөрийн тайланд бичнэ.

Жагсаалт 2. Bitmask-д суурилсан бүс шалгах функц

```c
static int check_region(region_t reg, int no, worker_report *rep) {
    unsigned mask = 0;
    int ok = 1;
    for (int k = 0; k < N; k++) {
        int r, c;
        cell_of(reg, no, k, &r, &c);
        int v = sudoku[r][c];
        if (v < 1 || v > 9) {
            ok = 0;
            add_issue(rep, reg, no, ISSUE_RANGE, v, r, c);
            continue;
        }
        if (mask & (1u << v)) {
            ok = 0;
            add_issue(rep, reg, no, ISSUE_DUP, v, r, c);
        }
        mask |= 1u << v;
    }
    if (mask != FULL_MASK) {              /* FULL_MASK = 0x3FE */
        ok = 0;
        for (int d = 1; d <= 9; d++)
            if (!(mask & (1u << d))) add_issue(rep, reg, no, ISSUE_MISSING, d, -1,
    -1);
    }
    return ok;
}
```

5.4 Worker функцууд ба төлөвлөгөө

Дөрвөн worker функц (`check_rows`, `check_columns`, `check_subgrid`, `check_all`) бүгд `void *fn(void *)` хэлбэртэй. Жишээ болгон мөр шалгагчийг харуулав. Урсгал нь үр дүнгээ

зөвхөн `valid[p->index]`-д бичиж, `pthread_exit(NULL)`-ээр дуусна.

Жагсаалт 3. Мөр шалгах worker урсгал

```
static void *check_rows(void *arg) {
    parameters *p = (parameters *)arg;
    int ok = 1;
    for (int r = p->row; r < p->row + p->count; r++)
        ok &= check_region(REG_ROW, r, &reports[p->index]);
    valid[p->index] = ok;          /* үр дүнг parent-д буцаах */
    pthread_exit(NULL);
}
```

`build_plan()` нь горимоос хамааран функцийн заагч, `row`, `column`, `count` утгатай `plan_item` массив үүсгэнэ. 11 горимд эхний хоёр бичлэг нь `count=9`-тэй мөр, баганын worker, 27 горимд `count=1`-тэй 18 бичлэг, хоёр горимд хоёуланд нь сүүлийн 9 бичлэг нь дэд торын worker байна. Ингэснээр стратеги нэмэх нь зөвхөн төлөвлөгөөг өөрчлөх асуудал болж, урсгал удирдах код өөрчлөгдөхгүй.

5.5 Урсгал үүсгэх ба хүлээх

`run_validation()` нь fork–join загварын цөм юм. `valid[]`-ийг эхлээд 0 болгосноор ямар нэг worker ажиллаагүй тохиолдолд ч үр дүн «буруу» болох аюулгүй анхдагч төлөвтэй (fail-safe). Хугацааг зөвхөн урсгал үүсгэх эхлэлээс бүх join дуусах хүртэл хэмждэг тул тайлангийн нэр бичих, `memset` зэрэг бэлтгэл ажил хэмжилтэд орохгүй.

Жагсаалт 4. Fork–join: урсгал үүсгэх, хүлээх, нэгтгэх (товчилсон)

```
for (int i = 0; i < n; i++) valid[i] = 0;    /* fail-safe */
clock_gettime(CLOCK_MONOTONIC, &t0);
for (int i = 0; i < n; i++) {
    parameters *data = malloc(sizeof(parameters));
    if (!data) { perror("malloc"); exit(2); }
    data->row = plan[i].row;      data->column = plan[i].column;
    data->count = plan[i].count;  data->index = i;
    params[i] = data;
    int rc = pthread_create(&tids[i], NULL, plan[i].fn, data);
    if (rc != 0) { fprintf(stderr, "pthread_create: %s\n", strerror(rc)); exit(2); }
}
for (int i = 0; i < n; i++) {
    int rc = pthread_join(tids[i], NULL);
    if (rc != 0) { fprintf(stderr, "pthread_join: %s\n", strerror(rc)); exit(2); }
    free(params[i]);
}
clock_gettime(CLOCK_MONOTONIC, &t1);
int all_ok = 1;
for (int i = 0; i < n; i++) if (valid[i] != 1) all_ok = 0;
```

5.6 Оролт, командын мөрийн интерфэйс

Оролт уншигч нь `| - + , ;` болон хоосон зайг алгасаж, `.` эсвэл `_`-ийг хоосон нүд гэж үзэн, `#`-ээс мөрийн төгсгөл хүртэлх тайлбарыг хасна. Файлын эхэнд UTF-8 BOM байж болно. Мөр бүрд 9 нүд эсвэл бүх 81 нүд нэг мөрөнд байх ёстой. Тоон аргументыг `atoi` биш `strtoll`-оор чанд шалгадаг тул `-t 11x`, `-e -3`, хэт том тоо зэрэг буруу утга 2 гэсэн гаралтын кодоор татгалзагдана.

Хүснэгт 5.2. Командын мөрийн сонголтууд

Сонголт	Тайлбар
`-f FILE`	Файл эсвэл `-` (stdin)-ээс 81 нүд унших
`-g SEED`	SEED-ээр давтагдах санамсаргүй зөв Sudoku үүсгэх (xorshift32, 0–4294967295)
`-e K`	K ширхэг санамсаргүй нүдийг өөр цифрээр солих (0–81)
`-s R,C,V`	[R,C] нүдийг V болгох, давтаж болно
`-t MODE`	Урсгалын стратеги: 1, 11 (анхдагч), 27
`-b REPS`	Гурван горимыг REPS удаа ажиллуулж дундаж, хамгийн бага, хамгийн их хугацааг харьцуулах
`-v`, `-q`	Урсгал бүрийн эхлэл/төгсгөлийг хэвлэх; зөвхөн эцсийн дүнг хэвлэх

5.7 HTTP гүүрийн гол хэсэг

Сервер нь самбарыг 9 мөр текст болгон C программын stdin руу дамжуулж, гаралтаас `valid[] = [...]` болон хугацааг тогтмол илэрхийллээр уншина. Worker-ийн тоо горимтой таарахгүй бол алдаа гэж үзнэ.

Жагсаалт 5. server.py: C хөдөлгүүрийг дуудах хэсэг

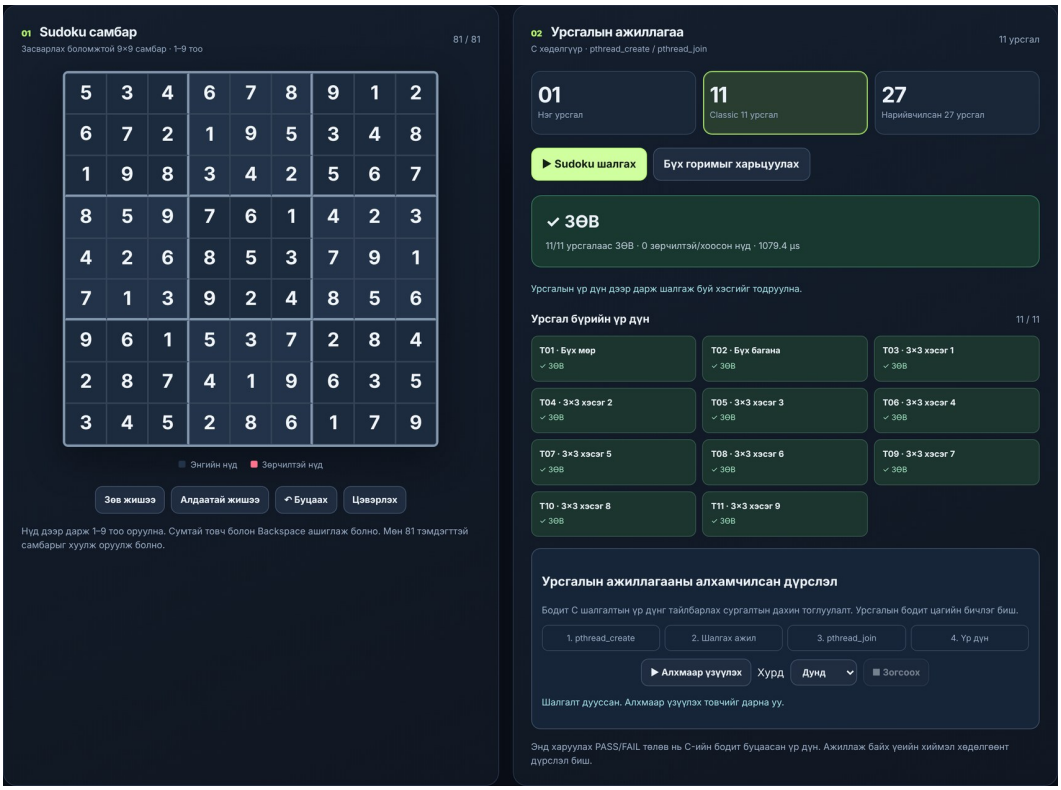
```
grid_text = '\n'.join(''.join(str(x) for x in board[r*9:(r+1)*9]) for r in range(9))
     + '\n'
run = subprocess.run([str(BINARY), '-f', '-', '-t', str(mode)], input=grid_text,
                     encoding='utf-8', errors='replace', capture_output=True,
                     timeout=8, cwd=ROOT)
if run.returncode not in (0, 1):
    raise RuntimeError(run.stderr.strip() or f'C engine exited with code {run.
    returncode}')
flags = [int(v) for v in VALID.search(run.stdout).group(1).split()]
if len(flags) != mode:
    raise RuntimeError('Worker-ийн үр дүнгийн тоо зөрлөө.')
```

6. Туршилт ба UI дүрслэл

Систем хоёр төрлийн интерфэйстэй. **Dashboard** нь бодит C хөдөлгүүрийн үр дүнг харуулдаг бол **симулятор** нь урсгалын хуваарийг ойлгуулах зорилготой загвар юм. Энэ хоёрыг хольж тайлбарлахгүй байх нь чухал: Dashboard дээрх PASS/FAIL ба микросекунд нь бодит хэмжилт, симулятор дээрх «tick» нь хийсвэр нэгж.

6.1 Dashboard: бодит хөдөлгүүрийн интерфэйс

Зураг 6.1-д Figure 4.26-аас өөр зөв самбарыг 11 урсгалаар шалгасан бодит дэлгэцийн зургийг үзүүлэв. Баруун талд урсгал бүрийн бодит `valid[i]` утга (`T01 · Бүх мөр` гэх мэт), эцсийн дүн, C-ийн хэмжсэн хугацаа харагдана.



Зураг 6.1. Dashboard: зөв самбарыг 11 урсгалаар шалгасан үр дүн (бодит дэлгэцийн зураг)

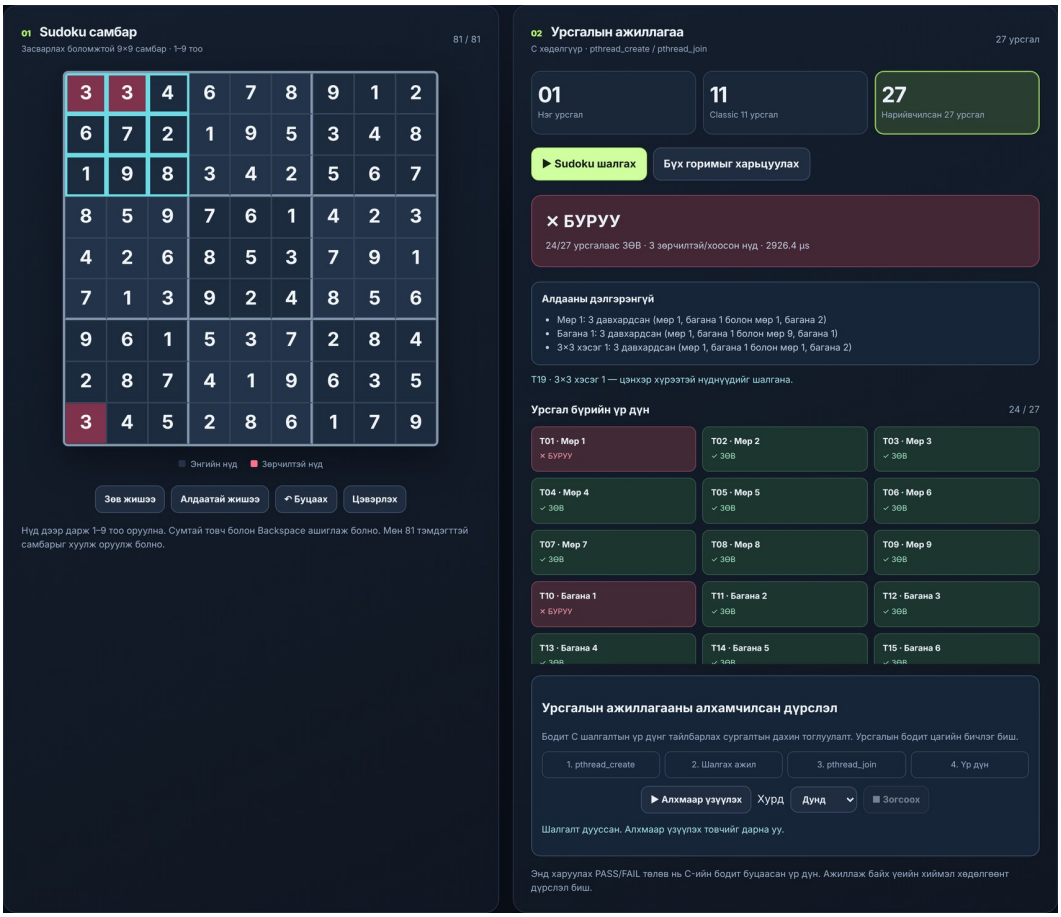
6.2 Юуг өөрчилж, юуг ажиглах вэ

Хүснэгт 6.1. Интерфэйс дээрх туршилтын хувьсагч ба ажиглалт

Юуг өөрчлөх	Юуг ажиглах	Хүлээгдэх үр дүн
«Алдаатай жишээ» эсвэл нэг нүдийг засах	Улаан нүд, БУРУУ болсон worker, «Алдааны дэлгэрэнгүй»	27 горимд яг 3 worker (мөр, багана, дэд тор) 0 буцаана
Горимыг $1 \to 11 \to 27$ болгох	Worker-ийн тоо, хугацаа (µs)	Дүгнэлт ижил; хугацаа урсгалын тоогоор өснө
Worker карт дээр дарах	Цэнхэр хүрээтэй нүд	Тухайн урсгалын хариуцах бүс тодорно
Нүд хоослох (Backspace)	«Хоосон нүд» оношлогоо	Хоосон нүд хамаарах бүх бүс буруу
«Хугацаа хэмжих» (3/10/30 давталт)	Медиан, хурдны харьцаа	1 урсгал хамгийн хурдан
Симулятор: цөм, үүсгэх зардал, квант, дараалал	Gantt, ашиглалт, контекст солилт, хурдатгал	Цөм нэмэхэд 27 урсгал $27c + 10$ tick-д ханана

6.3 Алдаатай ба бүрэн бус самбар

Зураг 6.2-д [1,1] нүдийг 3 болгосон самбарыг 27 урсгалаар шалгав. Мөр 1, багана 1, дэд тор 1-ийг хариуцсан T01, T10, T19 гэсэн гурван урсгал БУРУУ буцааж, бусад 24 нь ЗӨВ гэж буцаасан. T19 карт дээр дарахад дэд тор 1-ийн нүднүүд цэнхэр хүрээгээр тодорсон байна. Энэ нь нэг нүдний алдаа яг гурван бүсэд нөлөөлдөг гэсэн онолын хүлээлттэй тохирно.

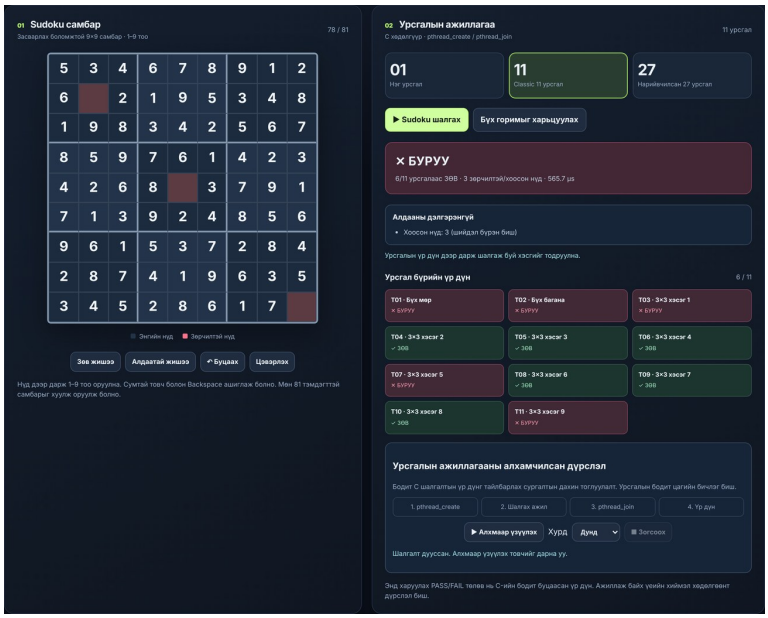


Зураг 6.2. Dashboard: нэг нүд буруу самбарыг 27 урсгалаар шалгаж, T19 урсгалын бүсийг тодруулсан нь

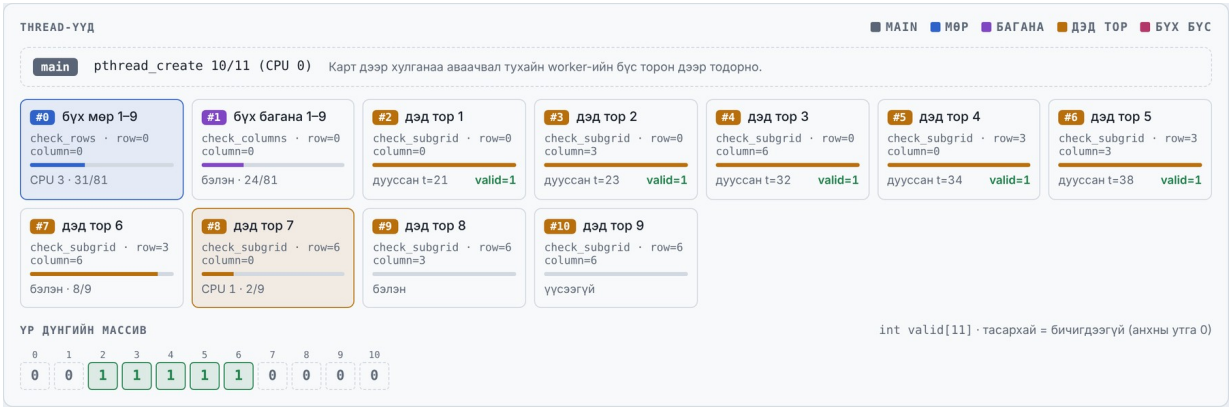
6.4 Сургалтын симулятор

`ui/sudoku_threads.html` нь C программын `build_plan()`-тай ижил төлөвлөгөөг JavaScript-ээр давтаж, олон цөмт CPU болон round-robin хуваарилагчийг tick-ээр загварчилна. 1 tick = нэг нүд шалгах ажил. Эх урсгал урсгал бүрийг үүсгэхэд c tick зарцуулна. Хэрэглэгч цөмийн тоо (1–8), үүсгэх зардал c (1–20), квант (2–64 эсвэл ∞), ready queue-ийн дараалал (FIFO/санамсаргүй)-ыг өөрчилж болно. Мөн санамсаргүй үүсгэгч нь C программын xorshift32-ын яг хуулбар тул ижил seed ижил самбар өгдөг бөгөөд C программд ижил туршилтыг давтах командыг автоматаар үүсгэнэ.

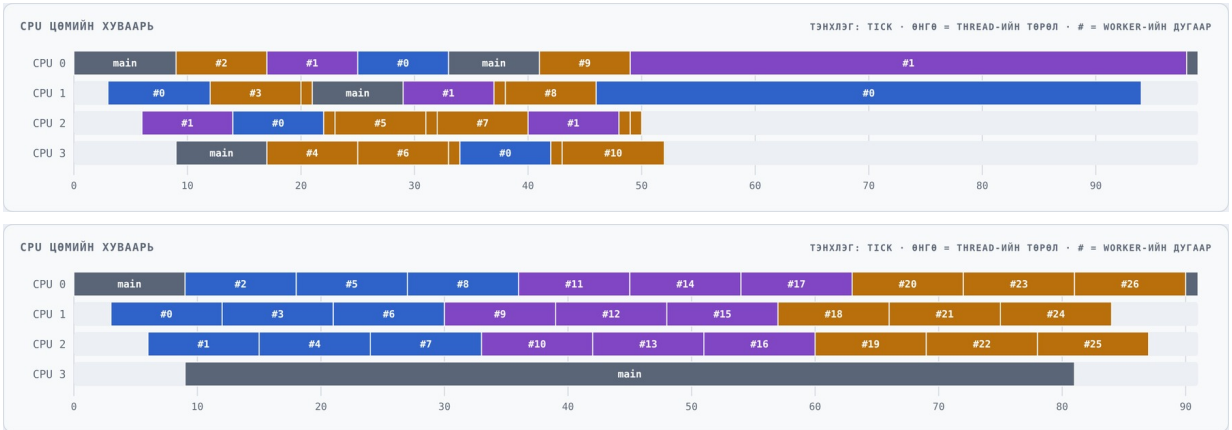
Зураг 6.5-ийн дээд хэсэгт 11 урсгалын тэнцвэргүй байдал тод харагдана: дэд торын богино worker-ууд эрт дуусах бол T0, T1 нь сүүл хүртэл ажиллана. Доод хэсэгт 27 урсгалын үед `main` урсгал 81 tick турш урсгал үүсгэсээр байгаа нь харагдана. Өөрөөр хэлбэл, эгзэгтэй зам нь worker-ийн ажил биш, харин эх урсгалын *дараалсан үүсгэлт* болж байна.



Зураг 6.3. Dashboard: 3 хоосон нүдтэй бүрэн бус самбар (11 урсгал). Хоосон нүд бүр буруу гэж тооцогдоно



Зураг 6.4. Симулятор: 11 урсгалын горимын 40-р tick. Дэд торын worker-ууд дуусч `valid[2..6]=1` болсон, T0 ба T1 үргэлжилж байна



Зураг 6.5. Симуляторын CPU цөмийн хуваарь (4 цөм, $c = 3$, квант 8): дээр 11 урсгал (99 tick), доор 27 урсгал (91 tick)

7. Туршилтын арга

Энэ бүлэгт туршилтын орчин, оролтын өгөгдөл, туршилт бүрийн арга болон хүлээгдэх үр дүнг тодорхойлно. Туршилтыг бусад хүн давтан хийх боломжтой байхаар бүх параметрийг тэмдэглэсэн.

7.1 Туршилтын орчин

Хүснэгт 7.1. Туршилтын орчин

Үзүүлэлт	Утга
Процессор	Intel Xeon (виртуал), 2 vCPU, цөм бүрд 1 урсгал
Санах ой	8 GB
Үйлдлийн систем	Linux 6.18 (x86_64), Ubuntu 24.04, glibc 2.39 (NPTL)
Хөрвүүлэгч	GCC 13.3.0, `-std=c11 -O2 -Wall -Wextra -pthread`
Sanitizer хувилбар	`-g -O1 -fsanitize=thread`; `-g -O1 -fsanitize=address,undefined`
Python / хөтөч	Python 3.13, Google Chrome 153 (headless, дэлгэцийн зураг авахад)
Цөм хязгаарлах	`taskset -c 0` (1 цөм) ба хязгааргүй (2 цөм)

7.2 Туршилтын өгөгдөл

Хүснэгт 7.2. Тестийн оролтын файлууд ба хүлээгдэх үр дүн

Файл	Агуулга	Код
`valid_fig426.txt`	Сурах бичгийн Figure 4.26, `|`, `-`, тайлбартай формат	0
`valid_compact.txt`, `valid_bom.txt`	81 цифр нэг мөрөнд; UTF-8 BOM-той	0
`invalid_one_cell.txt`	[1,1] нүд 6→2: мөр 1, багана 1, дэд тор 1 буруу	1
`invalid_subgrids_only.txt`	Латин квадрат $((r+c) \bmod 9)+1$: мөр, багана зөв, бүх дэд тор буруу	1
`invalid_columns_only.txt`	[1,1] ба [1,2]-ыг сольсон: зөвхөн багана 1, 2 буруу	1
`invalid_incomplete.txt`, `invalid_all_same.txt`	2 хоосон нүдтэй; бүх нүд 5	1
`bad_short.txt`, `bad_char.txt`, `bad_shape.txt`	80 нүд; үсэгтэй; мөрийн бүтэц буруу	2
`-g SEED [-e K]`	300 санамсаргүй зөв самбар, K = 1–5 алдаатай хувилбар	0 / 1

7.3 Туршилтын төлөвлөгөө

Хүснэгт 7.3. Туршилтууд, арга, хүлээгдэх үр дүн

ID	Туршилт	Арга	Хүлээгдэх үр дүн
E1	Функциональ зөв байдал	`run_tests.sh` хэсэг 1: 14 тохиолдол $\times$ 3 горим + 23 CLI тохиолдол	Гаралтын код 0/1/2 бүгд таарах
E2	Бүсийн тусгаарлалт	27 горимын `valid[]` мөрийг хүлээгдэх утгатай харьцуулах	Зөвхөн алдаатай бүсийн worker 0
E3	Стресс	300 seed $\times$ 3 горим $\times$ (зөв + алдаатай)	1800 тохиолдол бүгд зөв
E4	Синхрончлол, санах ой	TSan ба ASan+UBSan, 50 seed $\times$ 3 горим	Анхааруулга 0
E5	Дулаан хэмжилт	`-b 2000`, 3 удаа давтан; нэг процесс дотор	Урсгалын тоогоор хугацаа өснө
E6	Хүйтэн хэмжилт	Процесс бүрийг шинээр, горим бүрд 300 удаа	Тогтмол зардал нэмэгдэнэ
E7	Цөмийн тоо	E5, E6-г `taskset -c 0`-тэй давтах	Энэ ажилд 1 цөм хурдан байж болно
E8	HTTP ба API	100 POST/горим; буруу оролт, Host, Content-Type	Процесс эхлүүлэх зардал давамгайлна; 4xx хариу
E9	Загвар	Симуляторт цөм 1–8 ($c = 3$, квант 8)	27 урсгал $27c$-ээр хязгаарлагдана

7.4 Хэмжилтийн аргачлал

Хугацааг C программын дотор `CLOCK_MONOTONIC`-оор хэмжив. Хэмжилт нь урсгал үүсгэх, ажил, join, `malloc/free`-г багтаах бөгөөд процесс эхлүүлэх, оролт унших, хэвлэх ажлыг багтаахгүй. Хоёр төрлийн хэмжилт хийсэн:

- **Дулаан (warm):** `-b 2000` горим нь нэг процесс дотор 2000 удаа давтана. Энэ үед glibc урсгалын стекийг дахин ашиглах, кэш дүүрсэн байх зэрэг давуу талтай. Нийт 2000 давталтын дунджийг гурван бие даасан ажиллуулалтаар давтаж, дундаж $\pm$ стандарт хазайлтыг тооцов.
- **Хүйтэн (cold):** процесс бүр шинээр эхэлж, нэг л удаа шалгалт хийнэ. Энэ нь Dashboard болон HTTP гүүрийн бодит нөхцөлтэй ижил. Тархалт хазгай тул медиан, квартил хоорондын далайц (IQR), 95-р перцентилийг (p95) тайлагнав.

Нэг урсгал нэмэхэд гарах зардлыг (3.5) загварын b коэффициентоор, өөрөөр хэлбэл $(n,\ T)$ цэгүүдэд хамгийн бага квадратын аргаар тааруулсан шулууны налуугаар үнэлэв.

8. Үр дүн

Энэ бүлэгт 7-р бүлэгт тодорхойлсон E1–E9 туршилтын үр дүнг хүснэгт, график болон бодит дэлгэцийн зургаар танилцуулна. Үр дүнгийн шалтгаан, тайлбарыг 9-р бүлэгт хэлэлцэнэ.

8.1 Функциональ зөв байдал (E1–E4)

Хүснэгт 8.1. Автомат тестийн нэгтгэл (`make test`, нийт 6.5 секунд)

Хэсэг	Шалгалт	Амжилттай	Амжилтгүй
1. Файл, CLI тест (E1)	65	65	0
2. Бүсийн тусгаарлалт (E2)	5	5	0
3. Стресс, 300 seed (E3)	1 800	1 800	0
4a. ThreadSanitizer (E4)	150	150	0
4b. AddressSanitizer + UBSan (E4)	150	150	0
Нийт	**2 170**	**2 170**	**0**

Бүсийн тусгаарлалтын туршилт (Хүснэгт 8.2) нь зөвхөн алдаатай бүсийг хариуцсан worker 0 буцааж байгааг харуулна. Энэ нь урсгал бүр зөвхөн өөрийн бүсийг шалгаж, бусдын үр дүнд нөлөөлөхгүй байгааг баталгаажуулна.

Хүснэгт 8.2. 27 урсгалын горимын `valid[]` массив (мөр 1–9 | багана 1–9 | дэд тор 1–9)

Оролт	`valid[]`
Figure 4.26	`111111111 | 111111111 | 111111111`
Нэг нүд буруу	`011111111 | 011111111 | 011111111`
Латин квадрат	`111111111 | 111111111 | 000000000`
Зөвхөн багана	`111111111 | 001111111 | 111111111`
Латин квадрат (11 горим)	`1 1 | 0 0 0 0 0 0 0 0 0`

ThreadSanitizer [6] болон AddressSanitizer+UBSan [7]-ийн 300 ажиллуулалтад нэг ч анхааруулга гараагүй. Энэ нь 3.4-р хэсгийн Бернштейний нөхцөлд суурилсан «mutex шаардлагагүй» гэсэн дүгнэлтийг туршилтаар баталж байна. Зөв самбарыг `-v` горимоор ажиллуулахад урсгалуудын эхлэх, дуусах дараалал ажиллуулалт бүрд өөр байв (Жагсаалт 6). Жишээ нь, `#3` урсгал `#0`-ээс өмнө эхэлж, `#6` хамгийн сүүлд дууссан. Энэ нь урсгалуудыг цөмийн хуваарилагч бие даан хуваарилдаг болохыг харуулна. Гэсэн хэдий ч эцсийн дүн нь дарааллаас хамаардаггүй.

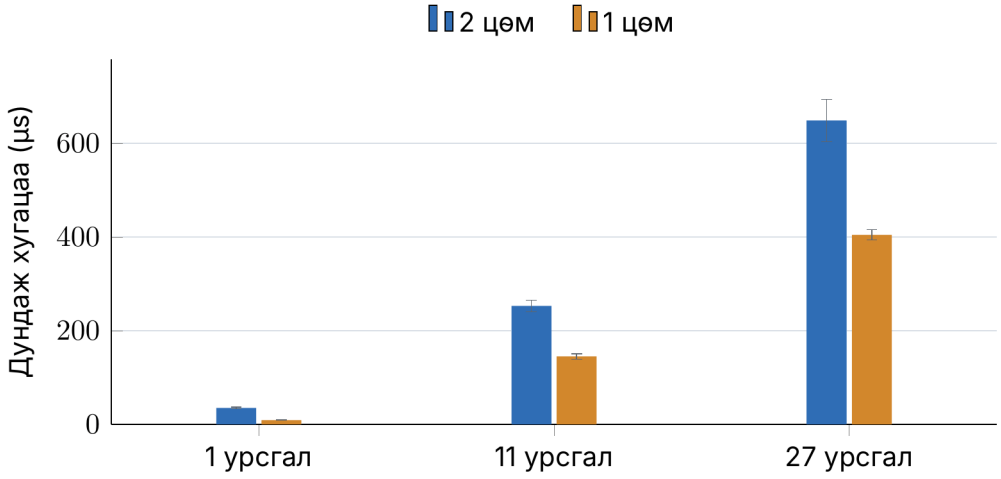
Жагсаалт 6. `./sudoku_validator -t 11 -v`-ийн гаралтын хэсэг: урсгалын дараалал детерминист биш

```
[main] pthread_create #0  rows     row=0 column=0 count=9
[main] pthread_create #1  columns  row=0 column=0 count=9
[main] pthread_create #2  subgrid  row=0 column=0 count=1
[main] pthread_create #3  subgrid  row=0 column=3 count=1
[main] pthread_create #4  subgrid  row=0 column=6 count=1
  [worker #3 ] эхэллээ   : дэд тор 2 (мөр 1-3, багана 4-6)
  [worker #3 ] дууслаа   : valid[3] = 1
  [worker #0 ] эхэллээ   : бүх мөр (1-9)
  [worker #0 ] дууслаа   : valid[0] = 1
  ...
  [worker #10] дууслаа   : valid[10] = 1
  [worker #6 ] эхэллээ   : дэд тор 5 (мөр 4-6, багана 4-6)
  [worker #6 ] дууслаа   : valid[6] = 1
[main] бүх 11 worker pthread_join-оор дууслаа
  valid[] = [1 1 1 1 1 1 1 1 1 1 1]  (11/11 worker зөв)
ДҮН: Судокугийн шийдэл ЗӨВ
```

8.2 Дулаан хэмжилт ба цөмийн тооны нөлөө (E5, E7)

Хүснэгт 8.3. Нэг процесс доторх хугацаа, `-b 2000`, 3 ажиллуулалтын дундаж ± стандарт хазайлт (µs)

	2 цөм		1 цөм (`taskset -c 0`)		Харьцаа T_n/T_1	
Горим	Дундаж	Хамгийн бага	Дундаж	Хамгийн бага	2 цөм	1 цөм
`-t 1`	35.07 ± 1.55	7.80	9.09 ± 0.28	7.42	1.00	1.00
`-t 11`	252.95 ± 12.32	162.67	144.96 ± 5.93	112.70	7.21	15.94
`-t 27`	649.00 ± 45.14	397.80	404.85 ± 10.95	313.04	18.51	44.52



Зураг 8.1. Дулаан хэмжилт: урсгалын тоо ба цөмийн тооны нөлөө (алдааны зураас: ±1 стандарт хазайлт)

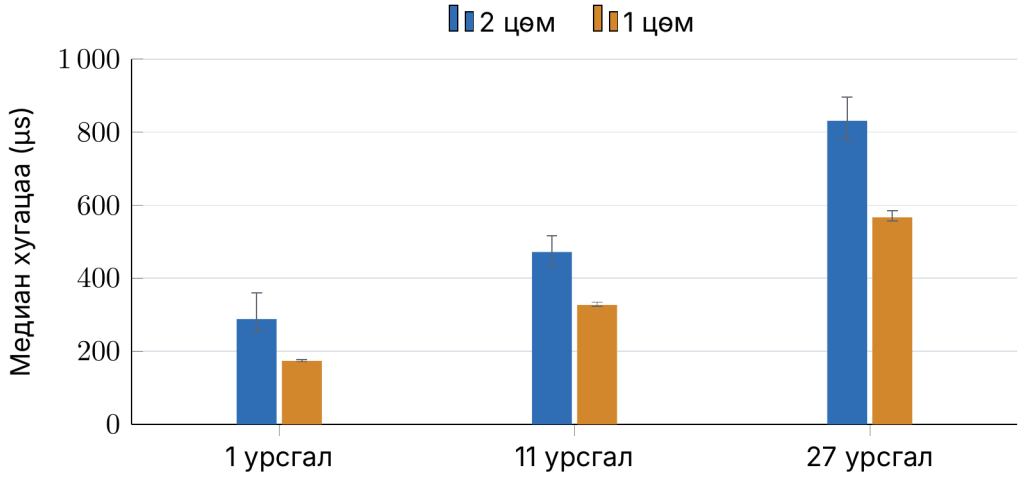
Ажиглалт 1. Урсгалын тоо нэмэгдэх тусам хугацаа бараг шугамаар өсөв. 2 цөм дээр 27 урсгал нь 1 урсгалаас 18.5 дахин удаан байлаа.

Ажиглалт 2. Энэ ажилд нэг цөм хоёр цөмөөс *хурдан* байв. 1 урсгалын горим 1 цөм дээр 9.1 µs, 2 цөм дээр 35.1 µs буюу 3.9 дахин ялгаатай.

8.3 Хүйтэн хэмжилт (E6)

Хүснэгт 8.4. Процесс бүрийг шинээр эхлүүлсэн хэмжилт, горим бүрд $n = 300$ (µs)

Цөм	Горим	Медиан	IQR	p95	Дундаж	Ст. хазайлт
2 цөм	`-t 1`	288.4	255.2 -- 360.2	440.0	316.2	79.6
	`-t 11`	472.1	429.7 -- 516.8	609.0	481.3	70.3
	`-t 27`	831.2	779.5 -- 895.7	1 100.4	853.2	117.4
1 цөм	`-t 1`	173.6	171.5 -- 177.0	199.0	176.7	12.1
	`-t 11`	327.1	323.1 -- 334.7	383.6	335.5	31.0
	`-t 27`	567.0	556.8 -- 585.0	765.0	588.9	66.4



Зураг 8.2. Хүйтэн хэмжилтийн медиан (алдааны зураас: 25 ба 75-р перцентиль)

Хүйтэн хэмжилтэд 1 урсгалын горим хүртэл 174–288 µs зарцуулсан нь дулаан хэмжилтийн 9–35 µs-ээс 8–19 дахин их. Харин урсгалын тоогоор өсөх хэсэг (налуу) дулаан хэмжилттэй ойролцоо байв.

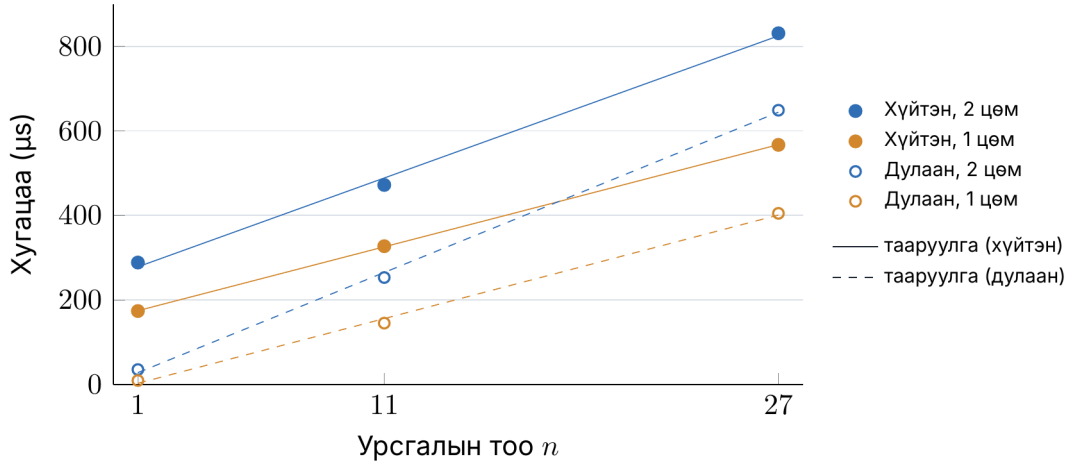
8.4 Нэг урсгалын нэмэлт зардлын үнэлгээ

(3.5) загварыг дөрвөн цуврал өгөгдөлд тааруулсан үр дүнг Хүснэгт 8.5, Зураг 8.3-д үзүүлэв.

Ажиглалт 3. Нэг урсгалын нэмэлт зардал $b \approx 15$ µs (1 цөм)–24 µs (2 цөм) бөгөөд дулаан, хүйтэн нөхцөлд ойролцоо байв. Харин 1 урсгалын горимын бүх ажил (243 нүд + 1 урсгал) 1 цөм дээр 9.1 µs. Иймд нэг нүд шалгах ажил $\leq 9.1/243 \approx 37$ нс, өөрөөр хэлбэл урсгал үүсгэх зардал нэг нүд шалгахаас *дор хаяж 400 дахин* их байна.

Хүснэгт 8.5. $T(n) \approx a + b\,n$ шугаман загварын коэффициент

Цуврал	a (µs)	b (µs / урсгал)
Дулаан, 2 цөм (дундаж)	4.0	23.7
Дулаан, 1 цөм (дундаж)	−12.8	15.3
Хүйтэн, 2 цөм (медиан)	257.2	21.0
Хүйтэн, 1 цөм (медиан)	159.3	15.1

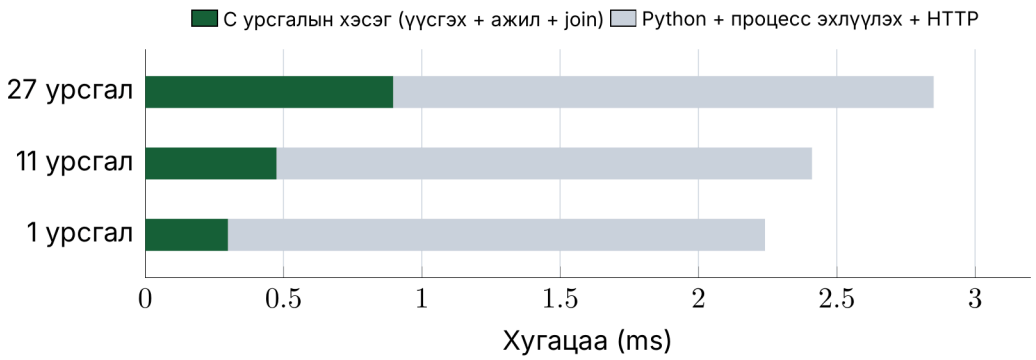


Зураг 8.3. Хугацаа урсгалын тооноос шугаман хамаарч байгаа нь (шулуун: хамгийн бага квадратын тааруулга)

8.5 HTTP-ээр дамжсан төгсгөл-хоорондын хугацаа (E8)

Хүснэгт 8.6. HTTP хүсэлтийн хугацаа, горим бүрд 100 хүсэлт (2 цөм)

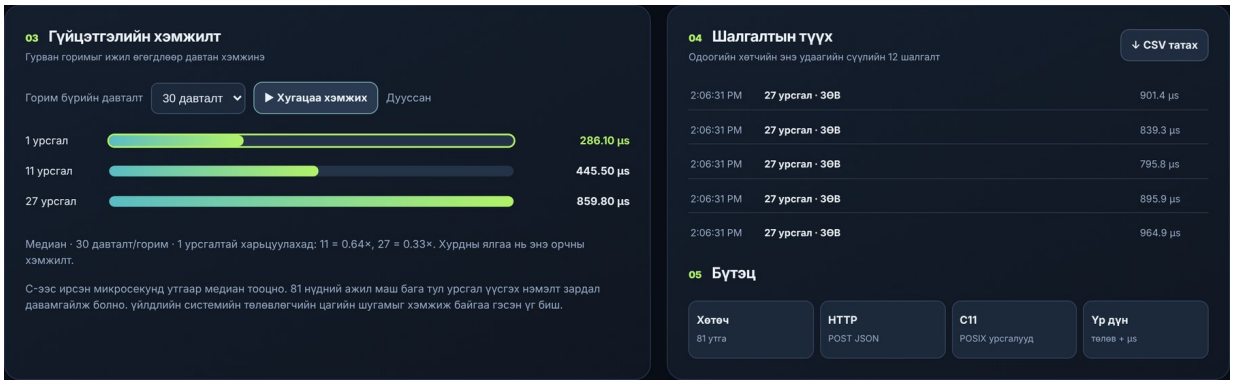
Горим	RTT медиан (ms)	RTT p95 (ms)	C хугацаа медиан (µs)	C-ийн эзлэх хувь
`-t 1`	2.24	2.58	298.8	13.4%
`-t 11`	2.41	2.64	473.9	19.7%
`-t 27`	2.85	3.08	895.8	31.4%



Зураг 8.4. HTTP хүсэлтийн хугацааны бүрэлдэхүүн (медиануудын зөрүүгээр ойролцоолсон)

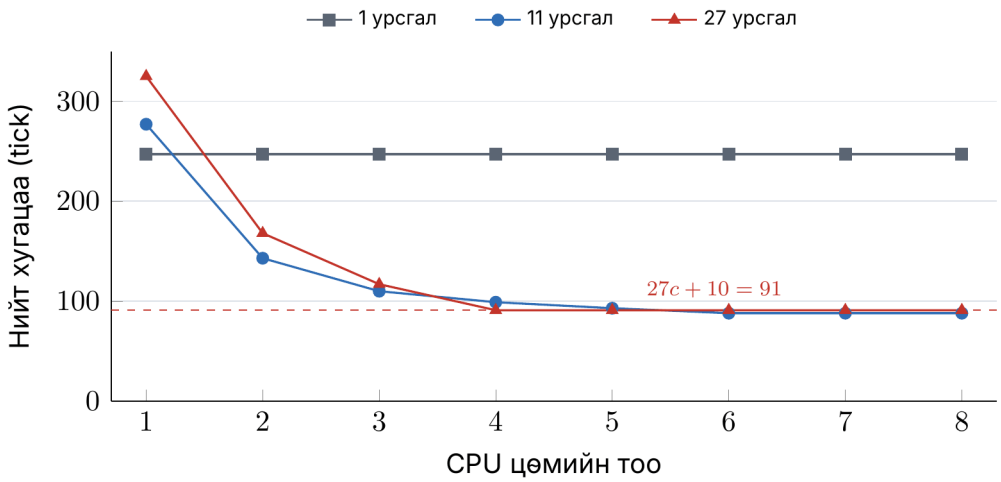
Урсгалаас бусад хэсэг (Python сервер, `subprocess`-ээр процесс эхлүүлэх, HTTP) горимоос үл хамааран **ойролцоогоор 1.94 ms** тогтмол байв. Буруу оролтын туршилтад сервер горим 5, 80 нүд, 10 гэсэн утгад 400; `text/plain`-д 415; хуурамч `Host` толгойд 403; `/server.py`, `/sudoku_validator.c`, `/tests/` замд 404 хариу өгсөн нь 4.5-р хэсгийн

зохиомжтой тохирно. Dashboard-ын «Хугацаа хэмжих» (30 давталт) функц 1, 11, 27 урсгалд 286.1, 445.5, 859.8 µs медиан гаргасан (Зураг 8.5). Энэ нь Хүснэгт 8.4-ийн хүйтэн хэмжилттэй нийцэж байна.



Зураг 8.5. Dashboard-ын гүйцэтгэлийн хэмжилт ба шалгалтын түүх (бодит дэлгэцийн зураг)

8.6 Симуляторын загварын үр дүн (E9)



Зураг 8.6. Симулятор: цөмийн тооноос хамаарах нийт хугацаа ($c = 3$ tick, квант 8, FIFO)

Хүснэгт 8.7. Симуляторын үр дүн, 4 цөм, $c = 3$, квант 8, FIFO

Стратеги	Хугацаа (tick)	CPU ашиглалт	Контекст солилт	Хурдатгал
1 урсгал	247	25%	2	1.00×
11 урсгал	99	70%	28	2.49×
27 урсгал	91	89%	26	2.71×

Загварт 1 урсгалын хугацаа $c+243+1 = 247$ tick, 27 урсгалын доод хязгаар $27c+9+1 = 91$ tick болж, симуляторын үр дүнтэй яг таарав. 11 урсгалын хувьд эгзэгтэй зам нь T0-ийн 81 нүд тул 6-аас дээш цөмд 88 tick-д ханасан.

9. Хэлэлцүүлэг

Энэ бүлэгт 8-р бүлгийн үр дүнг 3-р бүлгийн онолтой холбож тайлбарлан, хэрэгжилт болон туршилтын хязгаарлалт, сайжруулах боломжийг авч үзнэ.

9.1 Яагаад олон урсгал удаан байв?

(3.5) загварт хэмжилтийн утгыг орлуулбал $b \approx 15$–24 µs, $\max_i w_i$ нь хамгийн ихдээ 9 µs байна. 27 урсгалын үед урсгал үүсгэх зардал $27b \approx 400$–640 µs болох бол зэрэгцүүлснээр хэмнэх хугацаа 9 µs-ээс бага. Өөрөөр хэлбэл, Амдалын хуулийн хэлээр «зэрэгцээ хэсэг» нь нийт хугацааны маш бага хувийг эзэлж, урсгал нэмэх бүрт дараалсан хэсэг (эх урсгалын `pthread_create`) өсдөг. Иймд хурдатгал 1-ээс бага болно. Энэ нь «олон урсгал үргэлж хурдан» гэсэн түгээмэл буруу ойлголтыг няцаах тод жишээ юм.

Ашиг гарах босгыг тооцвол worker бүрийн ажил b-ээс хангалттай их буюу хэдэн арван микросекундээс урт байх ёстой. Нэг нүд $\sim$30 нс гэж үзвэл энэ нь worker бүр хэдэн мянган нүд шалгах ёстой гэсэн үг. Жишээ нь, олон мянган самбарыг багцаар шалгах эсвэл маш том хэмжээтэй тор шалгах үед урсгалын зэрэгцээ ажиллагаа ашигтай болно.

9.2 Яагаад нэг цөм хурдан байв?

Нэг цөмтэй үед шинэ урсгал эх урсгал `pthread_join`-д блоклогдох хүртэл ready queue-д хүлээж, дараа нь мөн цөм дээр шууд ажиллана. Хоёр цөмтэй үед харин хуваарилагч шинэ урсгалыг сул цөм рүү илгээдэг. Энэ нь (1) сул цөмийг сэрээх цөм хоорондын тасалдал (IPI) ба унтах төлөвөөс гарах хоцрогдол, (2) `sudoku`, `parameters`, `valid[]`-ийн кэшийн мөрийг нөгөө цөм рүү шилжүүлэх, (3) join хийх үед эх урсгалыг сэрээх нэмэлт тасалдал зэрэг зардал авчирна. Виртуал машинд эдгээр зардал илүү өндөр байдаг. 1 урсгалын горимд 1 цөм дээр 9.1 µs, 2 цөм дээр 35.1 µs байсан нь энэ тайлбартай нийцнэ. Мөн `valid[]`-ийн 27 элемент (108 байт) хоёр кэшийн мөрөнд багтдаг тул өөр цөм дээрх worker-ууд хуурамч хуваалцал (false sharing) үүсгэж болох боловч worker бүр нэг л удаа бичдэг тул нөлөө нь бага.

9.3 Хүйтэн ба дулаан хэмжилтийн ялгаа

Процесс дахь анхны `pthread_create` нь стекийн хэсгийг `mmap`-аар нөөцлөх, хамгаалалтын хуудас тохируулах, динамик холбогчийн тэмдэгтийг анх удаа шийдвэрлэх, хуудасны алдааг (page fault) боловсруулах зэрэг нэг удаагийн ажил хийдэг. Дулаан хэмжилтэд glibc өмнө нь дууссан урсгалын стекийг кэшлэн дахин ашигладаг тул эдгээр зардал арилдаг. Хүйтэн хэмжилтийн $a \approx 160$–260 µs тогтмол нь эдгээр нэг удаагийн зардлыг илтгэнэ. Харин налуу b хоёр нөхцөлд ойролцоо байгаа нь урсгал бүрт давтагддаг цөмийн зардал (`clone`, хуваарь, join) гол хүчин зүйл гэдгийг харуулна.

9.4 11 урсгалын стратегийн ачааллын тэнцвэргүй байдал

Сурах бичгийн 11 урсгалын хувилбарт T0 ба T1 нь тус бүр 81 нүд, бусад 9 урсгал 9 нүд шалгадаг. Иймд хязгааргүй олон цөмтэй байсан ч эгзэгтэй зам 81 нүднээс богиносохгүй, хамгийн ихдээ $243/81 = 3$ дахин хурдатгал гаргана. Симулятор энэ хязгаарыг 6-аас дээш цөмд 88 tick-д ханах байдлаар харуулсан. 27 урсгалын хувилбар ачааллын хувьд тэнцвэртэй ч эх урсгал 27 урсгалыг дараалан үүсгэх зардал эгзэгтэй зам болдог.

9.5 Симулятор ба бодит хэмжилтийн зөрүү

Симулятор 27 урсгалд 2.71× хурдатгал үзүүлсэн бол бодит хэмжилтэд 18.5 дахин удааширсан. Энэ зөрүүний шалтгаан нь загварын параметр юм. Симуляторт урсгал үүсгэх зардлыг $c = 3$ tick буюу 3 нүд шалгах ажилтай тэнцүү гэж тохируулсан бол бодит орчинд $c \gtrsim 400$ tick байна (Ажиглалт 3). Симуляторын гулсагчийн дээд хязгаар 20 tick байгаа нь бодит харьцааг хамарч чадахгүй. Иймд симуляторыг «урсгалын хуваарь хэрхэн ажилладаг»-ийг ойлгуулах хэрэгсэл гэж үзэх нь зөв, харин гүйцэтгэлийн дүгнэлтийг бодит хэмжилтэд үндэслэх хэрэгтэй.

9.6 Хязгаарлалт

- **Орчин:** хэмжилтийг 2 vCPU бүхий виртуал машинд хийсэн. Бодит олон цөмт компьютер дээр үнэмлэхүй утга өөр байх боловч чиг хандлага (урсгал нэмэхэд удаашрах) хадгалагдах төлөвтэй. Хэрэглэгч `./sudoku_validator -b 2000`-ээр өөрийн компьютер дээр давтан шалгаж болно.
- **Статистик:** C-ийн `-b` горим зөвхөн дундаж, хамгийн бага, хамгийн их утгыг гаргадаг тул дулаан хэмжилтийн медианыг тооцоогүй. Мөн Dashboard-ын 30 давталт бага түүвэр юм.
- **Логикийн давхардал:** `server.py` нь алдаатай нүдийг C-ээс биш Python дээр дахин тооцдог. Эцсийн дүгнэлт C-ээс гардаг ч хоёр хэрэгжилт зөрөх эрсдэлтэй.
- **`-v` горим:** worker доторх `printf` нь stdout-ын түгжээг ашигладаг тул хугацааг их хэмжээгээр нэмэгдүүлдэг (11 урсгалд 538.9 µs). Иймд хэмжилтийг `-v`-гүй хийсэн.
- **Алдаа оруулагч:** `-e K` нь нэг нүдийг өөрчлөхөд самбар заавал буруу болдог. Харин $K \geq 4$ үед тэгш өнцөгт хэлбэрийн 4 нүдний (a, b / b, a) солилтоор өөр зөв самбар үүсэх онолын маш бага магадлал бий. Стресс тестэд ийм тохиолдол гараагүй.
- **HTTP:** хүсэлт бүрд шинэ процесс үүсгэдэг тул 1.9 ms орчим тогтмол зардал гарна.

9.7 Сайжруулах санал

1. **Урсгалын сан (thread pool):** урсгалыг нэг удаа үүсгэж, олон самбарт дахин ашиглавал b зардал арилна.
2. **Багц шалгалт:** нэг хүсэлтээр олон самбар шалгавал ажил урсгалын зардлаас давж, зэрэгцээ ашиг гарна.

3. **`pthread_attr_setstacksize`:** жижиг стек (жишээ нь 64 KiB) ашиглаж хүйтэн зардлыг бууруулах.
4. **Кэшийн мөрөөр тэгшлэх:** `valid[]` элемент бүрийг 64 байтаар тэгшилж хуурамч хуваалцлыг арилгах.
5. **JSON гаралт:** C программ `--json` сонголтоор алдаатай нүдийг шууд гаргавал Python дахь давхар логик арилна.
6. **Статистик:** C-ийн benchmark-д медиан, перцентиль нэмэх.

10. Дүгнэлт

Энэхүү бие даалтаар сурах бичгийн Sudoku Solution Validator төслийг C11 болон POSIX Pthreads ашиглан бүрэн хэрэгжүүлж, шаардлага бүрийг (Ш1–Ш8) хангасан болохыг туршилтаар баталгаажуулав. Мөн 1, 11, 27 урсгалын стратеги, оношлогоо, гүйцэтгэлийн хэмжилт, localhost HTTP гүүр, вэб Dashboard болон CPU хуваарийн сургалтын симуляторыг нэмж хийсэн.

Нотолсон үр дүн:

- Хэрэгжилт зөв: 2170 автомат шалгалт 100% амжилттай, ThreadSanitizer ба AddressSanitizer+UBSan анхааруулга 0.
- Worker бүрийн бичих олонлог давхцахгүй тул mutex-гүй зэрэгцээ ажиллагаа аюулгүй гэдгийг Бернштейний нөхцөл болон TSan-аар давхар нотолсон.
- Урсгалын дараалал детерминист биш (`-v` гаралт) боловч эцсийн дүн дарааллаас хамаардаггүй.
- Бага ажилд урсгал нэмэх нь удаашруулдаг: 2 цөм дээр 1, 11, 27 урсгалын дундаж хугацаа 35.1, 253.0, 649.0 µs.
- Нэг урсгалын нэмэлт зардал ойролцоогоор 15–24 µs буюу нэг нүд шалгахаас дор хаяж 400 дахин их.
- Энэ ажилд 1 цөм 2 цөмөөс хурдан (1 урсгал: 9.1 ба 35.1 µs), учир нь цөм хоорондын сэрээлт, кэш шилжүүлэлтийн зардал ашгаас давна.
- 11 урсгалын стратеги ачааллын хувьд тэнцвэргүй (81 ба 9 нүд), онолын хурдатгалын дээд хязгаар нь 3.

Эндээс урсгалын тоог ажлын хэмжээ, урсгалын зардал, цөмийн тоог тооцож сонгох ёстой гэсэн практик дүгнэлт гарна. Цаашид урсгалын сан ба багц шалгалтыг хэрэгжүүлж, ажлын хэмжээ ба хурдатгалын хамаарлыг олон цөмт бодит техник хангамж дээр судлах нь зүйтэй.

Ашигласан материал

[1] A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018. Ch. 4 «Threads & Concurrency», Programming Project 1: Sudoku Solution Validator.

[2] IEEE Std 1003.1-2017 (POSIX.1-2017), *The Open Group Base Specifications Issue 7*: `pthread_create`, `pthread_join`; Base Definitions §4.12 «Memory Synchronization».

[3] Linux man-pages project, `pthreads(7)`, `pthread_create(3)`, `pthread_join(3)`, `clock_gettime(2)`, `taskset(1)`. `https://man7.org/linux/man-pages/`

[4] ISO/IEC 9899:2011, *Information technology — Programming languages — C*.

[5] G. M. Amdahl, «Validity of the single processor approach to achieving large scale computing capabilities», in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[6] K. Serebryany, T. Iskhodzhanov, «ThreadSanitizer: data race detection in practice», in *Proc. Workshop on Binary Instrumentation and Applications (WBIA)*, 2009.

[7] K. Serebryany, D. Bruening, A. Potapenko, D. Vyukov, «AddressSanitizer: A fast address sanity checker», in *Proc. USENIX ATC*, 2012.

[8] Python Software Foundation, *Python 3 Documentation*: `http.server`, `subprocess`, `json`. `https://docs.python.org/3/`

[9] U. Drepper, I. Molnar, «The Native POSIX Thread Library for Linux», Red Hat, 2005.

[10] Free Software Foundation, *GCC Manual*: Program Instrumentation Options (`-fsanitize`). `https://gcc.gnu.org/onlinedocs/`

[11] A. J. Bernstein, «Analysis of programs for parallel processing», *IEEE Trans. Electronic Computers*, vol. EC-15, no. 5, pp. 757–763, 1966.

[12] Төслийн эх код: Sudoku-Thread-Validator. `https://github.com/sumya4748-afk/Sudoku-Thread-Validator`

Хавсралт А. Ажлын төлөвлөгөө

А.1. Төслийн ерөнхий мэдээлэл

Талбар	Мэдээлэл
Хуваарилалтын код	IV-2-5
Төсөл	Programming Project 1 — Sudoku Solution Validator (*Operating System Concepts*, Бүлэг 4): 9×9 Sudoku-гийн шийдлийг 1, 11, 27 урсгалаар шалгах
Repository	`https://github.com/sumya4748-afk/Sudoku-Thread-Validator`
Хэл/сан/орчин	C11, POSIX Pthreads, GCC 13 + Make; Python 3 (`http.server`, `subprocess`); HTML/CSS/JavaScript; Linux, macOS, Windows (MSYS2 UCRT64); TSan, ASan+UBSan
Үндсэн дүрслэл	`dashboard.html`: бодит C хөдөлгүүрийн дүн, урсгал бүрийн `valid[i]`, алдаатай нүд, гүйцэтгэлийн хэмжилт; `ui/sudoku_threads.html`: CPU цөмийн хуваарийн (Gantt) симулятор

А.2. Гишүүдийн хариуцах ажил

Гишүүн	Хариуцах ажил	Commit/file
Сумъяасүрэн	Урсгалын цөм: `parameters` бүтэц, `check_rows`, `check_columns`, `check_subgrid`, `check_all` worker функц, `build_plan()` (1/11/27 стратеги), `run_validation()` (`pthread_create`, `pthread_join`, `valid[]`). *Тайлан: 2, 3-р бүлэг*	`sudoku_validator.c` (worker функц, `build_plan`, `run_validation`)
Мөнгөнтүлхүүр	Бүс шалгах bitmask алгоритм (`check_region`), оношлогоо (`worker_report`, `issue`), оролт уншигч (`load_grid`: BOM, тайлбар, мөрийн бүтэц), CLI (`getopt`, `parse_num`, `-g -e -s`), `-b` benchmark. *Тайлан: 5-р бүлэг*	`sudoku_validator.c` (`check_region`, `load_grid`, `generate`, `main`)
Тэмүүлэн	Автомат тест (`run_tests.sh`, 2170 шалгалт), тестийн оролтын файлууд, Makefile-ийн `tsan`/`asan` зорилт, гүйцэтгэлийн хэмжилт (E5–E8: `-b 2000`, 300 удаагийн хүйтэн хэмжилт, `taskset`). *Тайлан: 7, 8-р бүлэг*	`run_tests.sh`, `tests/*.txt`, `Makefile`
Ундрах	HTTP гүүр (`server.py`: JSON баталгаажуулалт, Host/Origin шалгалт, `Semaphore(4)`, timeout), нэг товшилтын эхлүүлэгч, Windows/macOS багц (PyInstaller, GitHub Actions). *Тайлан: 4-р бүлэг, Хавсралт Б*	`server.py`, `launch.py`, `launch_standalone.py`, `mac_app_launcher.py`, `START_*`, `build_windows.bat`
Марал-Эрдэнэ	Вэб интерфэйс (`dashboard.html`: самбар засах, горим, worker тодруулах, benchmark, CSV), сургалтын симулятор (`ui/sudoku_threads.html`: round-robin, Gantt), дэлгэцийн зураг, тайланг нэгтгэж засварлах. *Тайлан: Хураангуй, 1, 6, 9, 10-р бүлэг*	`dashboard.html`, `ui/sudoku_threads.html`, `sudoku_report.tex`

A.3. Гол тестүүд

Тест	Оролт/нөхцөл	Хүлээгдсэн үр дүн
1	`tests/valid_fig426.txt` (сурах бичгийн Figure 4.26), `-t 1`, `-t 11`, `-t 27`	Гаралтын код 0, «ДҮН: ЗӨВ»; `valid[]`-ийн бүх элемент 1
2	`tests/invalid_one_cell.txt`: [1,1] нүд 6→2, `-t 27`	Код 1; зөвхөн мөр 1, багана 1, дэд тор 1-ийн worker 0 буцаана (`valid[0]`, `valid[9]`, `valid[18]` = 0)
3	`tests/invalid_subgrids_only.txt`: Латин квадрат, `-t 27`	Код 1; 18 мөр/баганын worker 1, 9 дэд торын worker бүгд 0
4	`tests/invalid_incomplete.txt`: 2 хоосон нүдтэй самбар, 3 горим	Код 1; хоосон нүд хамаарах бүсүүд «1–9 биш / хоосон» гэж оношлогдоно
5	Буруу оролт: `bad_short.txt` (80 нүд), `bad_char.txt` (үсэг), `-t 5`, `-s 10,1,1`	Код 2; stderr-д оролтын алдааны тайлбар хэвлэгдэж, шалгалт эхлэхгүй

Дээрх 5 тест болон автомат шалгалт бүгд хүлээгдсэн үр дүнтэй таарсан (8.1-р хэсэг).